Folios — Architecture

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-05-30

This document describes how Folios is built. Intended for engineering reviewers, technical due diligence, or anyone evaluating the system beyond the product surface.

Stack at a glance

Layer	Technology
Frontend	React 18 + Vite 5 (SPA)
Distribution	PWA (vite-plugin-pwa, Workbox)
Hosting	Vercel (static + serverless functions)
Database	Supabase Postgres 15
Auth	Supabase Auth (email/password + TOTP MFA)
Realtime	Supabase Realtime (Postgres LISTEN/NOTIFY)
AI inference	Anthropic API (claude-haiku-4-5-20251001, Sonnet for synthesis)
Fonts	Self-hosted via @fontsource-variable
Styling	Inline styles + CSS custom properties
State	React hooks; no Redux/MobX
Testing	Vitest
Lint	ESLint + react-hooks plugin
CI	GitHub Actions (lint + build on PR)

System diagram

```
┌──────────────────────────────────────────────────────────┐
│                         BROWSER                          │
│  ┌────────────────────────────────────────────────────┐  │
│  │  React SPA (PWA)                                   │  │
│  │  - Hooks: useAccounts, useMeetings, useItems, ...  │  │
│  │  - Views: AccountsView, CadenceHub, GaugeView, ... │  │
│  │  - Pip: PipView, PipSummarizePreview, ...          │  │
│  │  - Service worker (offline shell)                  │  │
│  └────────────────────────────────────────────────────┘  │
└─────────┬───────────────────────────────────┬────────────┘
          │                                   │
          │ TLS                               │ TLS
          │                                   │
          ▼                                   ▼
  ┌────────────────┐                  ┌───────────────────┐
  │    VERCEL      │                  │    SUPABASE       │
  │ static + edge  │                  │ Postgres + Auth   │
  │                │                  │ + Realtime        │
  │  /api/pip      │                  │ + Storage         │
  │  /api/ask-pip  │                  │                   │
  └───────┬────────┘                  └───────────────────┘
          │
          │ TLS (server-side only)
          │
          ▼
  ┌───────────────────┐
  │  ANTHROPIC API    │
  │  claude-haiku-4-5│
  └───────────────────┘
```

The browser talks directly to Supabase for all CRUD operations and to Vercel for AI calls. No bespoke backend exists.

Frontend architecture

Routing

Folios uses URL-based view selection but does not use React Router. A simple `view` state in `App.jsx` drives which top-level component renders. This keeps the bundle smaller and the navigation logic explicit. URLs encode the active view + selected account; back/forward work via standard browser history.

Code splitting

Every top-level view is lazy-loaded via React.lazy + Suspense:

```
const AccountsView = React.lazy(() =>
import("./views/accounts/AccountsView"));
const GaugeView    = React.lazy(() =>
import("./views/gauge/GaugeView"));
// ...
```

Initial bundle is the shell + Pip + auth. Other views load on demand. This is what makes the post-deploy stale-chunk failure mode possible — and what makes the auto-recovery layer necessary (see `src/main.jsx` and `src/lib/errorLog.js`).

Data layer (hooks)

Every domain entity has a dedicated hook that owns its read + write surface:

- `useAccounts(userId, orgId)` — accounts CRUD
- `useMeetings(userId, accountId, orgId)` — meetings CRUD
- `useItems(userId, accountId, orgId)` — open items CRUD
- `useContacts(userId, accountId, orgId)` — contacts CRUD
- `useCadences(userId, accountId, orgId)` — cadences CRUD

- `useProjects(userId, accountId, orgId, childAccountIds)` — Gauge projects
- `useQuickTasks(userId)` — quick tasks tray
- `usePipAccountState(userId)` — Pip's per-account memory
- `usePipAssignmentHints(userId, accountId)` — Pip's assignment learning
- `usePipCorrections(userId, accountId)` — Pip's correction log
- `useGlossary(userId, orgId, accountId)` — Pip's glossary
- `useErrors(userId, opts)` — error log
- `useActivity(userId, opts)` — audit log
- `useOrg(userId)` — org membership

Each hook:

- Subscribes to a Supabase Realtime channel for its table
- Refetches on realtime change (debounced ~500ms)
- Exposes loading + error states
- Returns CRUD functions that fire optimistic writes via Supabase client (RLS-enforced server-side)

Components stay presentational — no data fetching in component code.

Theme system

Two themes (dark default, light spec'd) implemented via CSS custom properties on `<html data-theme="…">`. The C object in `src/lib/colors.js` exposes `var(--…)` references, so all inline `style={{ background: C.surface }}` consumers re-theme instantly with no remount.

Pre-mount theme application is done by an inline `<script>` in `index.html` to prevent flash-of-wrong-theme.

Animation

Pip's mood-driven animations are CSS-only (keyframes + state classes via `PipStateProvider` context). The animated mark glyphs use a shared rAF engine in `Mark.jsx`. Both respect `prefers-reduced-motion`.

PWA

Service worker generated by Workbox via `vite-plugin-pwa`:

- `skipWaiting: true`, `clientsClaim: true`, `cleanupOutdatedCaches: true`
- Precaches static assets + shell HTML
- Two redundant auto-update paths in `src/main.jsx`:
  - `controllerchange` event listener (the "right" way)
  - Version polling every 3 min + on visibility change (fallback when SW is stuck)

- Both converge on `triggerReload()` with 60-second cooldown to prevent reload loops during mid-deploy edge-flip-flop.

Mobile

- Mobile-first layout with `useBreakpoint()` hook (900px threshold)
- Mobile shell: top header + scrollable content + fixed bottom nav
- All inputs ≥16px font (prevents iOS Safari auto-zoom)
- Safe-area insets respected (`env(safe-area-inset-*)`)
- Sheet-style modal on mobile vs. centered on desktop

Backend architecture

Supabase Postgres

All CRUD goes directly browser → Supabase via the JS client. Auth is handled by Supabase Auth; every request carries the user's JWT.

Row-Level Security

Every user-data table has RLS policies. Example:

```sql
alter table folio_accounts enable row level security;

create policy "accounts_select_own" on folio_accounts
  for select using (auth.uid() = user_id);

create policy "accounts_insert_own" on folio_accounts
  for insert with check (auth.uid() = user_id);
```

For org-shared tables, the policy expands to org membership lookup.

Realtime

Every domain hook subscribes to its table's realtime channel:

```js
supabase.channel("folio_accounts:" + userId)
  .on("postgres_changes", { event: "*", schema: "public",
       table: "folio_accounts", filter: "user_id=eq." + userId },
      debouncedRefetch)
  .subscribe();
```

Debounce prevents thrash; refetch ensures the local cache stays authoritative. Multi-device sync works because every device of the same user subscribes to the same channel.

Migrations

SQL migrations live in `supabase/*.sql`. Run manually against production via Supabase Dashboard or `psql`. Canonical schema is in `supabase/schema.sql` (kept in sync as the source of truth).

Migration convention:

- Idempotent (`create table if not exists`, `alter table ... add column if not exists`)
- Single-block (the whole migration runs as one SQL statement)
- Safe to re-run

Pip pipeline

```
User input → classifyIntent → mode-specific prompt → Anthropic API →
  ↓                                                      ↓
streaming response →                                 tool calls →
  ↓                                                      ↓
UI updates (PipView)                                 executeTool →
                                                     Supabase writes
```

Intent classification

`classifyIntent(text)` (in `src/lib/pipIntent.js`) inspects the user's question and routes it to one of:

- **chat** — conversational answer
- **action** — should result in a tool call (add item, set follow-up, etc.)

- **deterministic** — can be answered locally without an API call (e.g., "how many open items do I have?")

Modes

`callPipApi(messages, context, opts)` is the single network entrypoint. `opts.mode` selects the system prompt and context shape:

- `chat` — general Q&A with full context
- `brief` — pre-call brief for one account
- `summary` — meeting summarize with structured plan output (uses 4-block stacked prompt caching)
- `extract` — short-form action extraction from quick notes
- `compress` — distills correction log into lessons_learned paragraph

Tool calls

For action-mode requests, Pip can return tool_use blocks. `executeTool` in `src/lib/pipExecutor.js` dispatches to the right hook function (addItem, setFollowUp, closeItem, etc.). Tool calls are categorized:

- **Frictionless** — auto-execute (e.g., adding a quick task user asked for)
- **Confirm-required** — surfaced as PipActionCard for user approval

Cost optimization

- 4 cache breakpoints in summary mode (system / glossary / roster / items+tasks). Multi-call sessions get ~70% cache hit.
- Trivial draft skip: drafts <100 chars never call the API.
- Per-call telemetry written to `folio_pip_usage`.
- Rate limit: 20 req/min per user, enforced in `/api/pip`.

Streaming

Streaming responses use Anthropic's SSE format, parsed in `src/lib/pipStream.js`. Each chunk fires an `onDelta(text)` callback that progressively updates the UI.

Learning loop (V2 brain)

Three persistent tables feed Pip's learning:

- `pip_correction_log` — every user correction (last 10 read back per summarize call)
- `pip_account_state.lessons_learned` — periodically compressed paragraph distilled from corrections (~every 5 meetings)
- `pip_assignment_hints` — who Pip should route each kind of task to
- `pip_glossary` — user-taught vocabulary

See `docs/ai-governance.md` for the full learning-loop description.

Build & deploy

Local dev

```
npm install
npm run dev     # Vite dev server on :5173
```

`.env.local` carries `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY`. Anthropic key lives only in Vercel env vars.

Production build

```
npm run build  # Vite production build to dist/
```

Vercel deploys on every push to the configured production branch (`claude/build-folio-desktop-app-XzvZ5`). Builds are isolated per branch; only the production branch pushes count toward Vercel's deployment limit.

Migrations

Run manually against production Supabase via the Dashboard SQL editor or `psql` . Migrations are idempotent so re-runs are safe.

Secrets

- `ANTHROPIC_API_KEY` — Vercel env var (production + preview)
- `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` — public, shipped to client by design
- Supabase service-role key never used in client code; only in local maintenance scripts run by the developer with their own credentials

---

Multi-tenancy readiness

Even at single-user pilot, the architecture supports multi-tenant team mode without restructure:

- `folio_orgs` table for orgs
- `folio_org_members` for user-to-org mapping with role
- Every user-data table carries `org_id` (or is linked via account which carries it)

- RLS policies branch to check org membership for shared resources
- Org-wide assignment hints (account_id = null) so Pip can learn team-level patterns
- Activity log scoped per-org for owners

When team mode lights up, the changes are mostly UI (invite flow, member management, role-based visibility), not schema.

Observability

Client-side error capture

`src/lib/errorLog.js` exposes `logError(type, message, opts)`. Three sources:

- `installGlobalErrorHandlers()` (window.onerror + unhandledrejection)
- `ErrorBoundary.componentDidCatch` for React render errors
- Explicit `logError` calls from network/fetch helpers

Captured errors land in `folio_errors` (RLS-scoped per user). Visible in Settings → Diagnostics.

Self-healing

`looksLikeChunkReload(message, stack)` detects stale-chunk Lazy import failures and triggers `triggerReload()` immediately, with the 60s cooldown. Errors of this type are logged as `chunk_reload` with `resolved: true` so they don't clutter the unresolved list.

Performance

`timed(label, fn)` wraps hot-path operations and logs duration. Currently logs to console; future plan is to push to a perf table.

Cost

`folio_pip_usage` tracks every Pip API call (tokens, mode, cache hit flag, timestamp). Per-user cost dashboard in Settings → Pip Usage.

Key architectural decisions & tradeoffs

"No bespoke backend"

Decision: all CRUD goes browser → Supabase directly. Only AI calls go through a Vercel serverless function.

Tradeoff: Faster development, fewer moving parts, single layer of RLS enforcement (no chance of a custom backend forgetting to apply the right scope). Cost: complex queries that span tables are harder to express as a single RPC; we lean on multi-fetch + client-side joining.

"Inline styles + CSS variables"

Decision: no styled-components, no Tailwind, no CSS modules. Just inline styles consuming CSS custom properties.

Tradeoff: Smaller bundle, no runtime style overhead, instant re-theme via CSS variable swap. Cost: shared style patterns are slightly more verbose; we offset with `src/lib/colors.js` (`C` object) for token reuse.

"Hooks-first data layer"

Decision: every entity has a dedicated hook; components are presentational.

Tradeoff: Clear boundaries, easy to test data logic in isolation, straightforward realtime wiring per table. Cost: components that need multiple entities (e.g., AccountDetail) end up with many hook calls; we accept the verbosity for the clarity.

"Optimistic writes via Supabase client"

Decision: no explicit mutation library (no React Query, no SWR mutations). Writes go through Supabase JS client directly with realtime echoing the update back.

Tradeoff: Simpler mental model, fewer dependencies. Cost: no built-in retry-with-backoff for mutations; we add it manually in `src/lib/net.js` where needed.

"Pip is a thin proxy"

Decision: `/api/pip` is a thin auth + rate-limit + prompt-shape proxy to Anthropic. No conversation state stored server-side; client sends the full message history every call.

Tradeoff: Stateless, easy to scale. Cost: prompt size grows with history; we mitigate with prompt caching + per-mode context shaping.

File layout (top level)

```
/api                      Vercel serverless functions (Pip proxy)
/docs                     Presentation docs (this directory)
/scripts                  Local maintenance scripts (seed data, etc.)
/src
  /components             Reusable UI primitives
  /hooks                  Domain data hooks (one per entity)
  /layout                 App shell (MobileLayout, DesktopLayout)
  /lib                    Cross-cutting modules (colors, pip, errorLog,
...)
  /views                  Top-level pages, lazy-loaded
/supabase                 SQL migrations and canonical schema
CLAUDE.md                 Development context, queue, rules (internal)
vercel.json               Deploy config + cache headers
vite.config.js            Build + PWA config
index.html                Shell HTML + theme bootstrap + global CSS
```

Contact

For architecture questions or technical due diligence: chris.vasconcellos97@gmail.com